Visualização de Dados (parte 02)

Análise de Dados Financeiros e Econômicos com o R

Marcelo S. Perlin

EA-UFRGS

01/10/2026

Sumário

- Mapas com o R
- Programando com o `ggplot2`
- Resumo da aula

Mapas com o R

ADFER - UFRGS

Introdução

> a representação de dados em mapas é uma das técnicas de visualização de dados mais utilizadas pela mídia jornalística;

- a abordagem de camadas é intuitiva (uma camada para a representação visual da área geográfica, e outra para os pontos e preenchimentos)
- R oferece um ecossistema de pacotes baseados no `ggplot2` que facilitam a construção de mapas

Exemplo de mapa



Localização de consulados e embaixadas americanas no Brasil

Pacote geobr (Pereira and Goncalves 2023)

- arquivos de dados geográficos contendo o território nacional são disponibilizados no site do IBGE.
- O pacote `geobr` (Pereira and Goncalves 2023) (IPEA) é uma API para acesso aos dados do IBGE.

O formato SF (*simple features*)

- O formato *SF* foi criado pela *Open Geospatial Consortium* e é eficiente para a representação de mapas geográficos.
- No R, um mapa SF é um `tibble` com uma coluna especial de **geometria**, que guarda os polígonos (áreas) ou pontos junto com os metadados.
- O `ggplot2` desenha esses objetos com a camada `geom_sf()`.

Exemplo geobr (1)

- Code

```
Rows: 1
Columns: 2
$ year     <dbl> 2019
$ geometry <MULTIPOLYGON [°]> MULTIPOLYGON (((-49.70495
-...
```

Exemplo geobr (2)

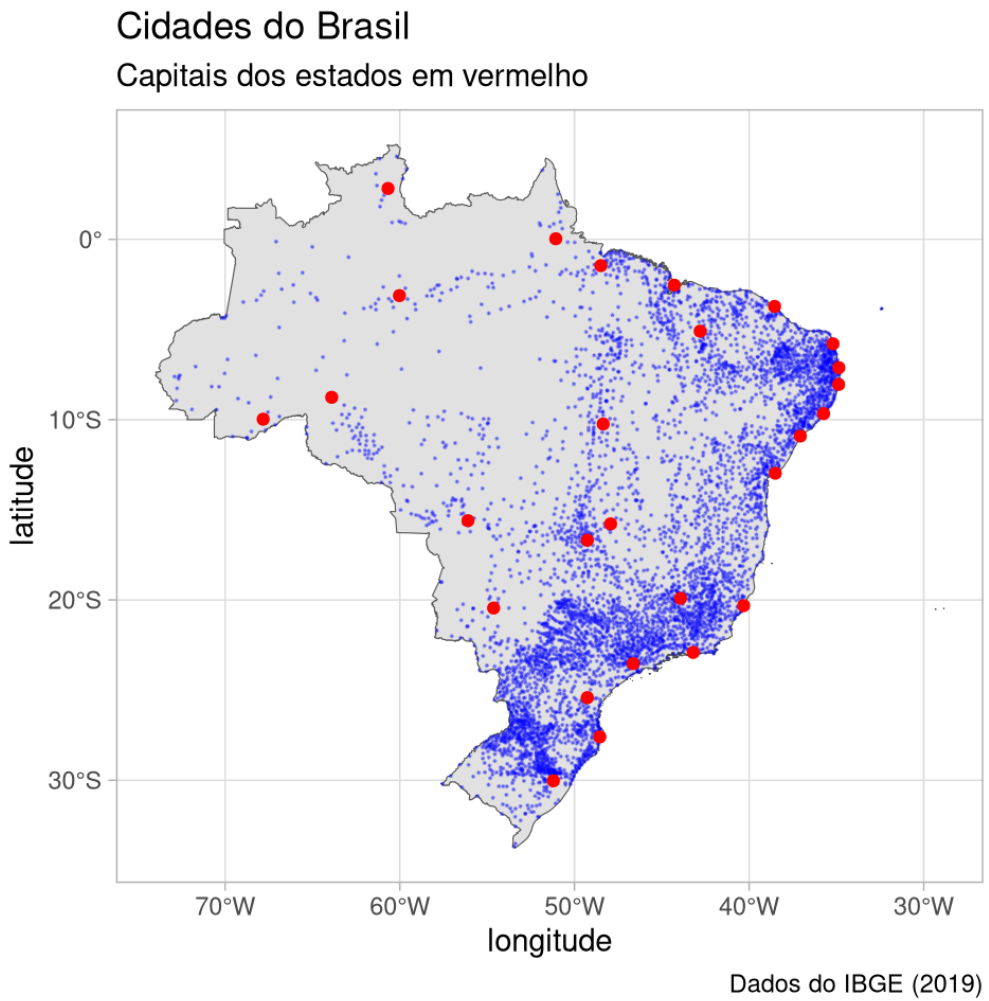
- Code



Mapa do Brasil

Exemplo geobr – cidades e capitais

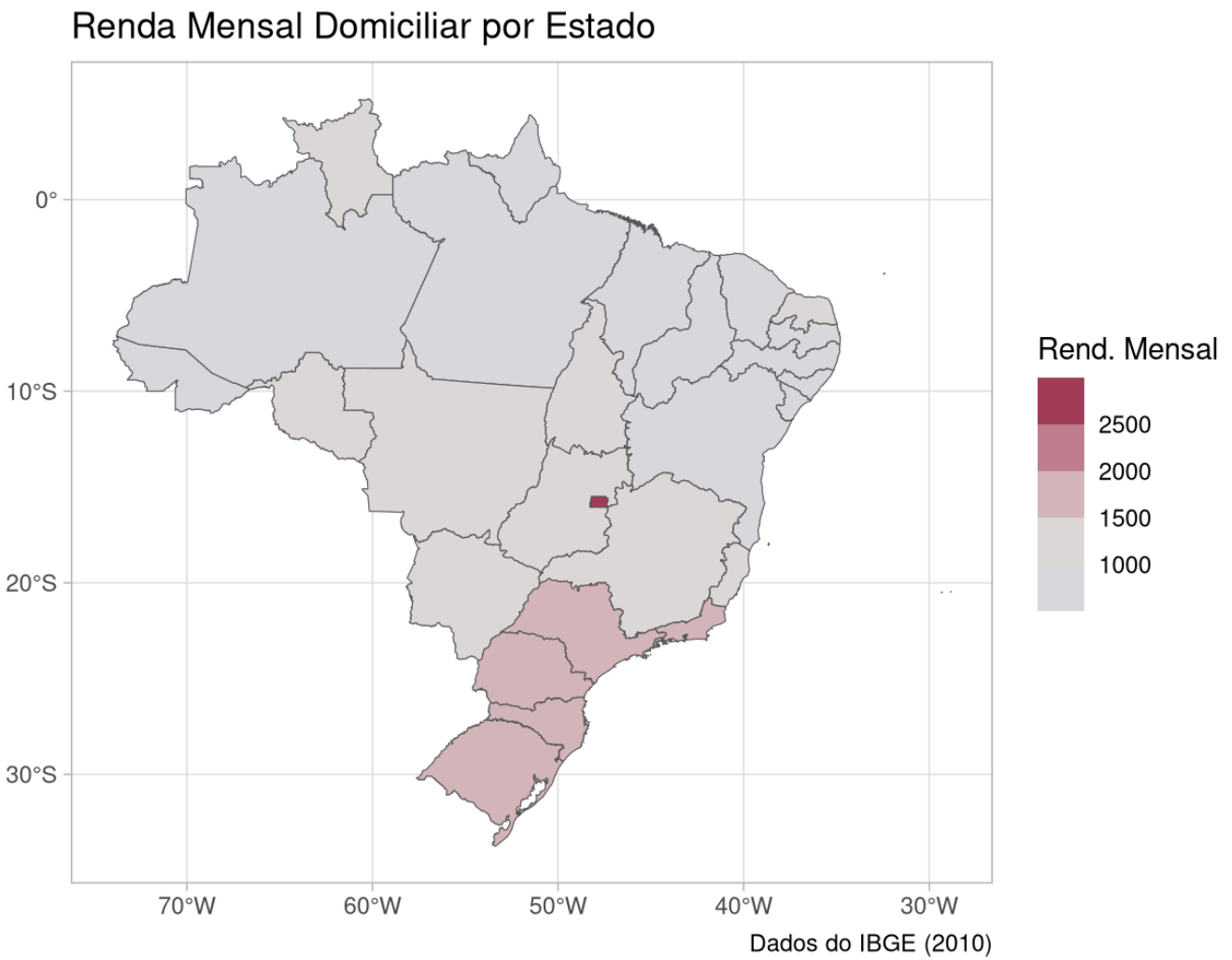
- Code



Mapa de cidades e capitais do Brasil

Exemplo geobr – Rendimento mensal domiciliar

- Code



Pacote ggmap (Kahle et al. 2023)

- API online para a importação de mapas do Google Maps e Stamen Maps.
- Disponibiliza mapas atualizados do mundo inteiro!

- **Google maps exige registro de projeto e cartão de crédito!**

Registrando um projeto no Google

1. Registre-se no Google através de um email. Caso já tenha uma conta, pule esse passo e faça o login no *browser* de preferência.
2. Acesse o site https://console.cloud.google.com/ e clique em “New Project”, definindo o nome do projeto e a localização.
3. Ative o serviço de “Google Maps” para o projeto criado anteriormente.
4. Recolha a chave na seção “Credentials”

Usando o API

```
1 library(ggmap)
2
3 google_key <- "YOUR_KEY_HERE"
4
5 register_google(
6   key = google_key,
7   account_type = 'standard'
8 )
```

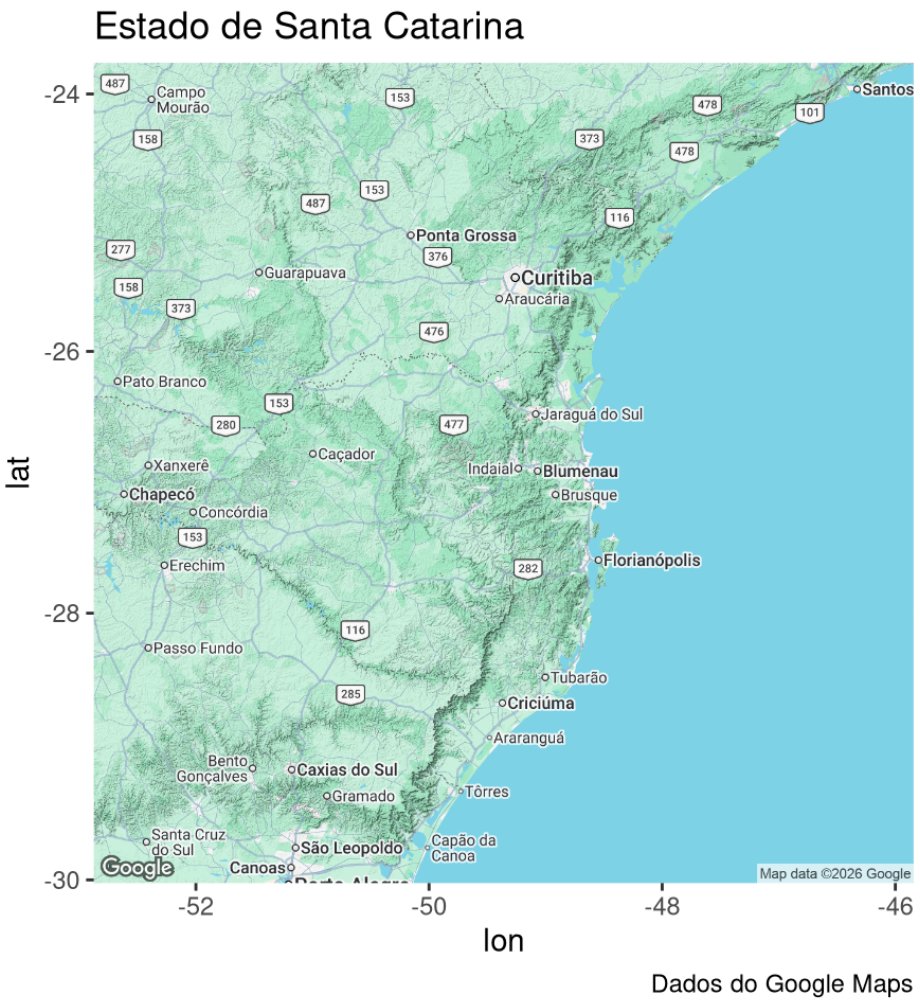
Após o registro da chave, teste a chamada ao API com um comando simples:

```
1 ggmap::geocode("Porto Alegre RS")
```

```
# A tibble: 1 × 2
    lon   lat
  <dbl> <dbl>
1 -51.2 -30.0
```

Utilizando ggmap

- Code



Exemplo de mapa para o estado de Santa Catarina

Codificação geográfica

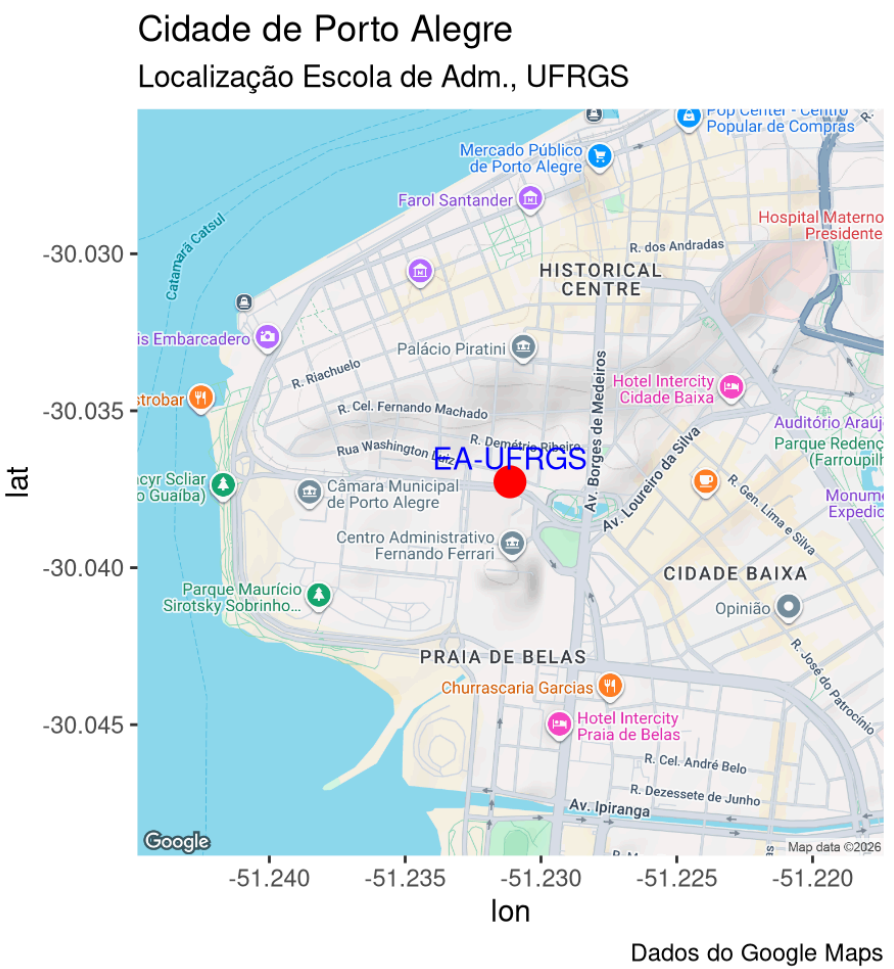
- codificação geográfica é a busca de latitude e longitude a partir de um texto (*string*)

```
1 ea_ufrgs <- ggmap::geocode("Escola de Administração UF
2
3 ea_ufrgs
```

```
# A tibble: 1 × 2
    lon   lat
  <dbl> <dbl>
1 -51.2 -30.0
```

Com a latitude e longitude do local em mãos, fica fácil de mostrar no mapa o ponto de interesse:

▶ Code



Programando com o ggplot2

Programação com o `ggplot2`

> A plataforma R fornece um ambiente computacional completo para automatizar qualquer tarefa de visualização de dados.

Aqui aprenderemos:

- Criação e exportação de figuras via funções;
- Teste de temas para figuras;

Criando figuras via funções

- A saída de **ggplot2::ggplot()** é um objeto do R como qualquer outro
- Podemos manipular esse objeto facilmente!
- Imagine o seguinte problema:
  - precisa criar um gráfico do preço do índice SP500 para anos diferentes, onde cada ano terá um gráfico individual, salvo em arquivo separado.

Solução

1. criar função com parâmetros de interesse
2. aplicar função a anos diferentes

```
plot_year <- function(this_year, df_yf) {

  df_year <- df_yf |>
    filter(
      lubridate::year(ref_date) == this_year
    )

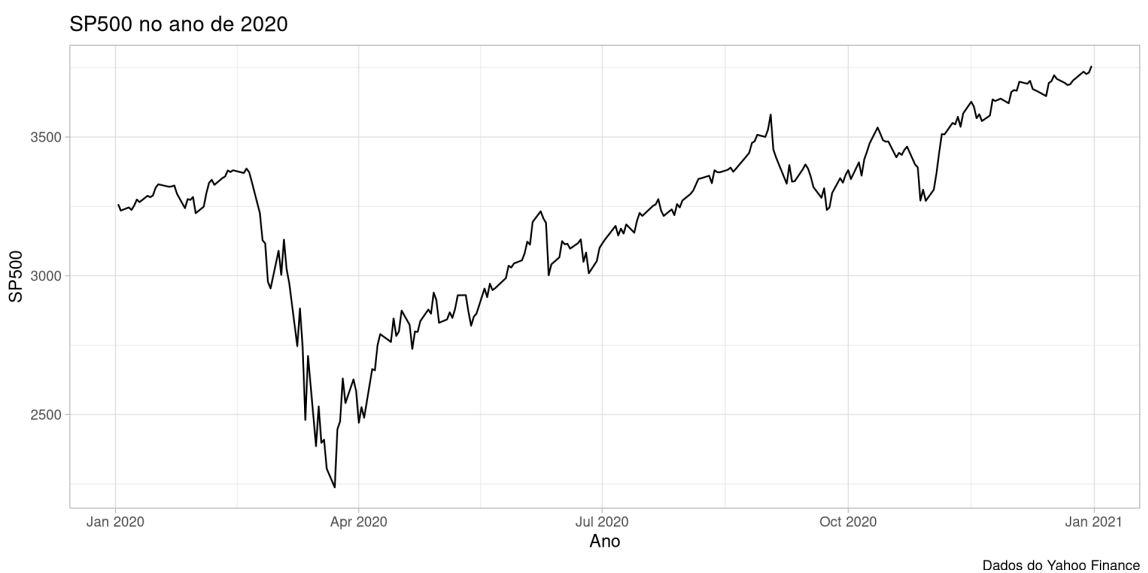
  p <- ggplot(df_year,
              aes(x = ref_date,
                  y = price_adjusted)) +
    geom_line() +
    labs(x = "Ano",
         y = "SP500",
```

Para testar a função, vamos importar os dados do SP500 e criar o gráfico para o ano de 2020:

```
df_yf <- vdr::data_import(
  "Chapter04-sp500-1950-2022.csv"
)

p <- plot_year(2020, df_yf)

p
```



SP500 para o ano 2020

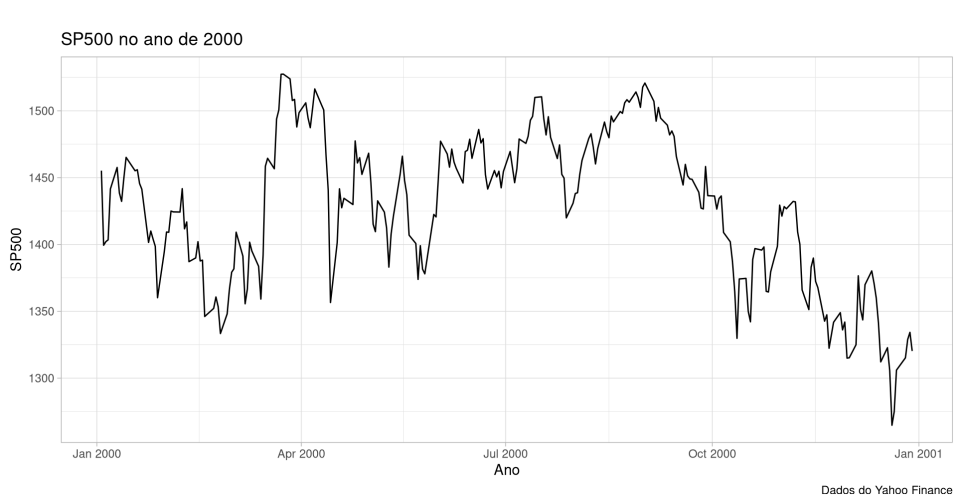
Aplicando a um vetor de anos:

```
years_to_plot <- 2000:2020

l_p <- purrr::map(
  .x = years_to_plot,
  .f = plot_year,
  df_yf = df_yf
)
```

O primeiro elemento de `l_p` é o gráfico para o ano de 2000:

```
l_p[[1]]
```



SP500 para o ano 2000

Iterando com `purrr` (vs. `for`)

- `purrr::map()` aplica uma função a cada elemento de um vetor e devolve uma **lista** com os resultados.
- É equivalente a um laço `for`, porém mais conciso e sem a necessidade de "pré-alocar" a lista:

```
# equivalente usando for
l_p <- vector("list", length(years_to_plot))

for (i in seq_along(years_to_plot)) {
  l_p[[i]] <- plot_year(years_to_plot[i], df_yf)
}
```

Agora que temos 1) a função que cria o gráfico por ano e 2) um conjunto de anos, podemos melhorar o código adicionando o processo de exportação das figuras.

```
plot_year_and_export <- function(this_year,
                                 df_yf,
                                 path_to_save = "figs"

  p <- plot_year(this_year, df_yf)

  fs::dir_create(path_to_save)

  file_to_save <- fs::path(
    path_to_save,
    paste0("SP500_", this_year, ".png")
  )

```

Aplicando a função:

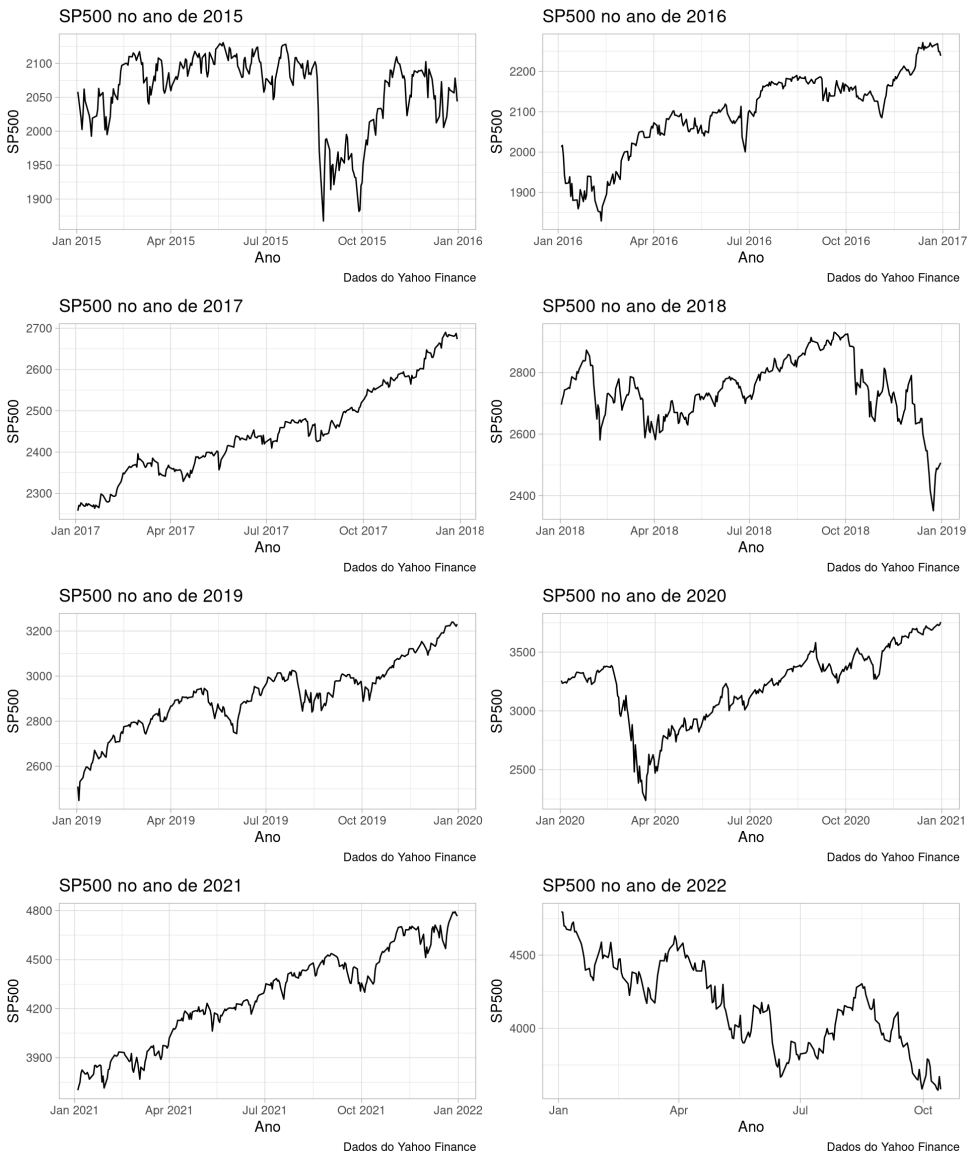
```
years_to_plot <- 2015:2022
dir_out <- fs::path_temp('sp500-by-year')

l_p <- purrr::map(
  .x = years_to_plot,
  .f = plot_year_and_export,
  path_to_save = dir_out,
  df_yf = df_yf
)

basename(fs::dir_ls(dir_out))
```

```
[1] "SP500_2015.png" "SP500_2016.png" "SP500_2017.png"
"SP500_2018.png"
[5] "SP500_2019.png" "SP500_2020.png" "SP500_2021.png"
"SP500_2022.png"
```

Resultado



Figuras com o índice SP500 por ano

Testando temas

Como sabemos, módulo **ggplot2** (Wickham et al. 2026) e **ggthemes** (Arnold 2026) oferecem diversos temas incluindo:

- **ggplot2::theme_bw()**
- **ggplot2::theme_grey()**
- **ggthemes::theme_excel()**
- **ggthemes::theme_economist()**

Primeiro criamos a função com tema:

```
plot_with_theme <- function(this_theme_fct, p) {

  p <- p + this_theme_fct()

  return(p)
}
```

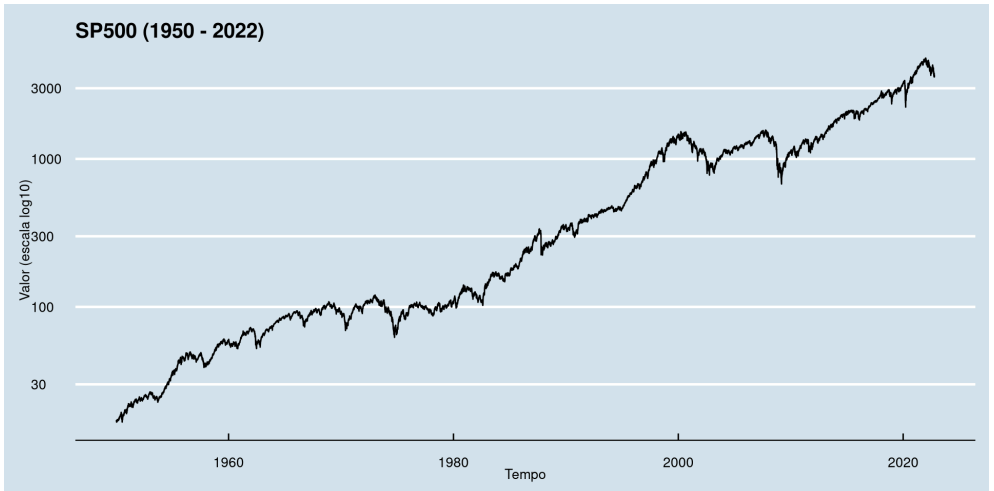
E agora testamos para o mesmo gráfico de preços e a função de tema **theme_economist()** :

```
p <- ggplot(df_yf,
            aes(x = ref_d
                y = price
  geom_line() +
  labs(x = 'Tempo',
       y = "Valor (escala
       title = "SP500 (19
  scale_y_continuous(trar



p <- plot_with_theme(ggth

p
```



Agora, vamos criar uma nova função que irá exportar o resultado em arquivos no formato png.

Note que passamos a **própria função de tema** (e não o seu nome como *string*): assim evitamos `eval(parse())` e deixamos o código mais seguro e idiomático.

```
plot_with_theme_and_export <- function(this_theme_fct,
                                       theme_label,
                                       p,
                                       path_to_save) {

  p <- plot_with_theme(this_theme_fct, p)

  fs::dir_create(path_to_save)

  p <- p +
    labs(subtitle = paste0("Tema = ", theme_label))

  file_to_save <- fs::path(
```

Aplicação da função:

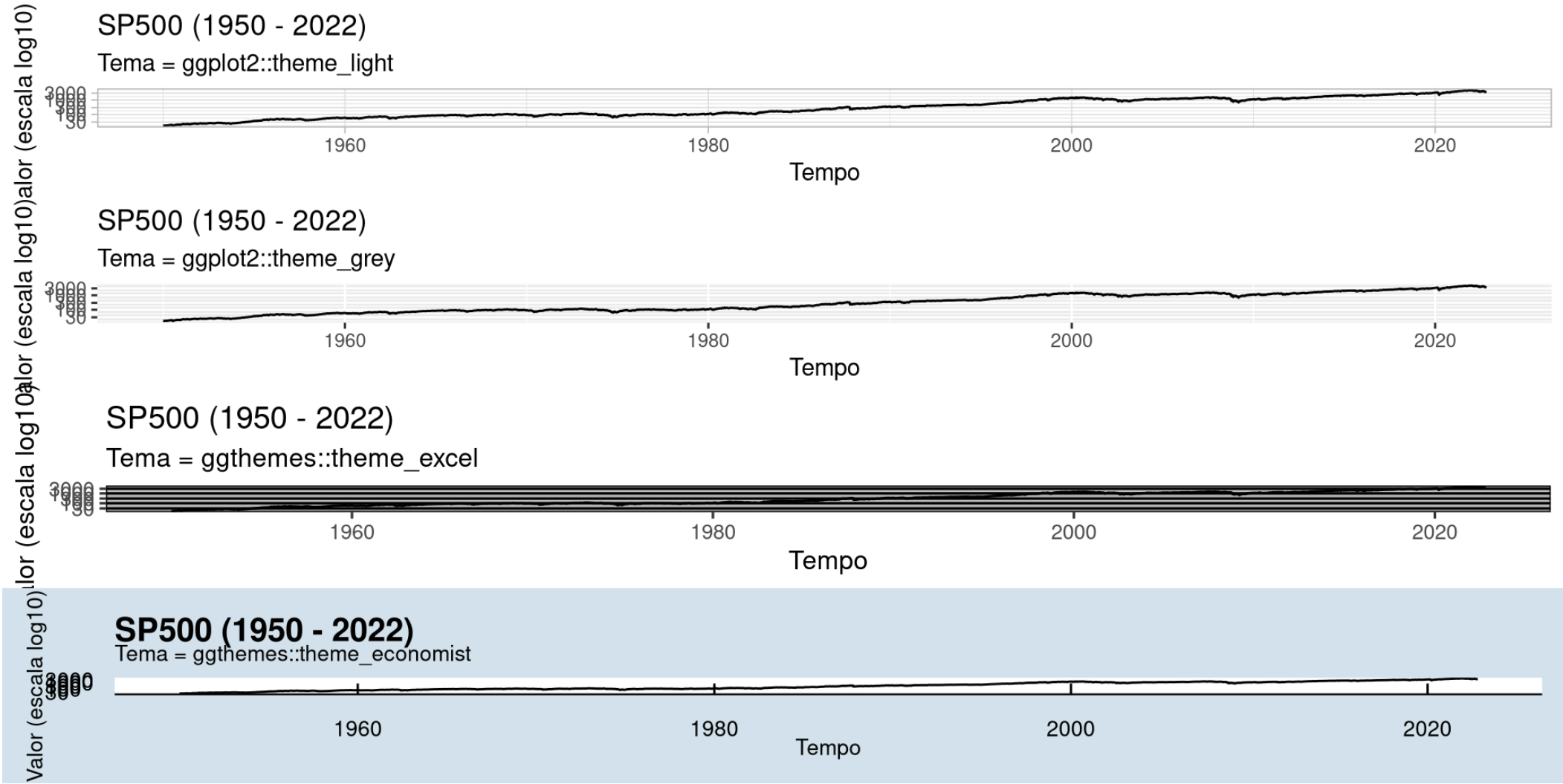
```
themes <- list(
  "ggplot2::theme_light"       = ggplot2::theme_light,
  "ggplot2::theme_grey"        = ggplot2::theme_grey,
  "ggthemes::theme_excel"      = ggthemes::theme_excel,
  "ggthemes::theme_economist" = ggthemes::theme_econor
)

dir_out <- fs::path_temp("testing-themes")

l_p <- purrr::imap(
  .x = themes,
  .f = function(fct, nm) plot_with_theme_and_export(
    this_theme_fct = fct,
```

```
[1] "SP500-1950-2022_ggplot2_theme_grey.png"
[2] "SP500-1950-2022_ggplot2_theme_light.png"
```

Resultado



Figuras com o índice SP500 por tema de gráfico

Resumo da aula

O que vimos

- Mapas no R com `geobr` (dados oficiais do IBGE) e `ggmap` (mapas do Google).
- Objetos no formato *SF* são desenhados com a camada `geom_sf()`.
- Figuras do `ggplot2` são objetos: podem ser guardadas em listas e manipuladas.
- `purrr::map()` e `purrr::imap()` aplicam funções a vários elementos e automatizam a produção de figuras.
- Funções com parâmetros permitem testar temas e exportar várias figuras de uma vez.

Referências

Arnold, Jeffrey B. 2026. *Ggthemes: Extra Themes, Scales and Geoms for Ggplot2*. https://jrnold.github.io/ggthemes/.

Kahle, David, Hadley Wickham, and Scott Jackson. 2023. *Ggmap: Spatial Visualization with Ggplot2*. https://github.com/dkahle/ggmap.

Pereira, Rafael H. M., and Caio Nogueira Goncalves. 2023. *Geobr: Download Official Spatial Data Sets of Brazil*. https://github.com/ipeaGIT/geobr.

Wickham, Hadley, Winston Chang, Lionel Henry, et al. 2026. *Ggplot2: Create Elegant Data Visualisations Using the Grammar of Graphics*. https://ggplot2.tidyverse.org.